CHAPTER 1

Introduction to language models

Human language and software engineering source code represent discrete symbolic systems governed by intricate, multi-level structural constraints spanning phonology, syntax, semantics, and pragmatics. For decades, computational linguistics attempted to capture these phenomena through hand-crafted rule bases and formal context-free grammars. However, natural language is inherently ambiguous, productive, and elastic: speakers continuously coin new expressions, tolerate grammatical ellipses, and convey contextual connotations that elude rigid deterministic parsers.

To construct computational systems capable of understanding, completing, and generating coherent text, modern computer science reframes language processing from deterministic rule parsing into a discrete stochastic process. By treating words or subword tokens as discrete random variables, language modeling formalizes the quantification of joint probability distributions over arbitrary token sequences. Through the unconditional factorization of the chain rule of probability, this joint estimation problem collapses into sequential conditional next-token prediction. This formulation—estimating the conditional distribution over a vocabulary given preceding context—constitutes the foundational principle uniting early statistical Markov models with modern deep neural foundation architectures. In this chapter, we establish the mathematical foundations of probabilistic language modeling, formalize the trade-offs of classical n-gram representations, analyze the breakdown of discrete frequency counting on unseen data, derive statistical smoothing algorithms, and establish information-theoretic metrics for sequence evaluation.

1.1 The sequence prediction problem and probabilistic language modeling

At its core, a language model formalizes the mechanism by which computational systems evaluate the fluency, plausibility, and likelihood of word sequences. In natural language processing and modern AI-assisted software engineering, we model language and source code not through brittle deterministic rules, but as a discrete stochastic process governed by formal probability distributions.

Let $\mathcal{V} = \{w_1, w_2, \ldots, w_V\}$ denote a finite vocabulary of $V = |\mathcal{V}|$ discrete tokens, such as words, subwords, or punctuation symbols. An arbitrary natural language sentence, code snippet, or document of length N is represented as an ordered sequence of discrete random variables $W = (w_1, w_2, \ldots, w_N) \in \mathcal{V}^N$.

Definition 1.1: *Statistical language model*

A *statistical language model* is a formal probability distribution $\mathbb{P}$ defined over the set of all possible discrete sequences $\mathcal{S} = \bigcup_{N=1}^{\infty} \mathcal{V}^N$. For any specific sequence $W = (w_1, \ldots, w_N)$, the language model assigns a scalar probability $\mathbb{P}(W) \in [0, 1]$ satisfying the Kolmogorov probability axioms:

$$\mathbb{P}(W) \geq 0 \quad \forall W \in \mathcal{S}, \quad \text{and} \quad \sum_{S \in \mathcal{S}} \mathbb{P}(S) = 1. \tag{1.1}$$

The model maps the syntactic and semantic conventions of human language into real values on the unit interval. Under a well-calibrated distribution, fluent and grammatically coherent sentences receive significant probability mass, as in:

$$\mathbb{P}(\text{“this”, “is”, “a”, “reasonable”, “sentence”}) = 0.1.$$

In contrast, syntactically valid but semantically unusual or rare utterances receive noticeably lower values:

$$\mathbb{P}(\text{“this”, “is”, “a”, “purple”, “sentence”}) = 0.01,$$

while ungrammatical, corrupted, or disjoint sequences receive negligible probabilities:

$$\mathbb{P}(\text{“this”, “are”, “a”, “reasonable”, “sentence”}) = 0.001.$$

Evaluating the joint probability $\mathbb{P}(w_1, \ldots, w_N)$ directly over the combinatorial space $\mathcal{V}^N$ is computationally intractable: observing every conceivable multi-word sequence in a training corpus is impossible. To structure this estimation problem, we invoke the fundamental chain rule of probability.

Definition 1.2: *Chain rule decomposition of language models*

For any sequence of random variables $(w_1, w_2, \ldots, w_N)$, the joint probability factors identically and unconditionally into an ordered product of conditional next-token probabilities:

$$\begin{aligned} \mathbb{P}(w_1, w_2, \ldots, w_N) &= \mathbb{P}(w_1) \cdot \mathbb{P}(w_2 \mid w_1) \cdot \mathbb{P}(w_3 \mid w_1, w_2) \cdots \mathbb{P}(w_N \mid w_1, \ldots, w_{N-1}) \\ &= \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_1, \ldots, w_{t-1}) = \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_{<t}), \end{aligned} \tag{1.2}$$

where $w_{<t} = (w_1, \ldots, w_{t-1})$ represents the prefix history preceding timestep t. For the

initial token $t = 1$, the conditioning prefix is the empty sequence $\emptyset$, yielding $\mathbb{P}(w_1 \mid w_{<1}) = \mathbb{P}(w_1)$.

Applying Equation (**??**) to our running example yields the sequential factorization:

$$\begin{aligned}\mathbb{P}(\text{this, is, a, reasonable, sentence}) = \mathbb{P}(\text{this}) \\ \cdot \mathbb{P}(\text{is} \mid \text{this}) \\ \cdot \mathbb{P}(\text{a} \mid \text{this, is}) \\ \cdot \mathbb{P}(\text{reasonable} \mid \text{this, is, a}) \\ \cdot \mathbb{P}(\text{sentence} \mid \text{this, is, a, reasonable}).\end{aligned}$$

This mathematical decomposition marks a fundamental conceptual shift in computational linguistics. Instead of attempting to score complete, static documents simultaneously, the modeling objective collapses entirely into sequential conditional next-word estimation: given history $w_{<t}$, the model estimates the categorical probability distribution $\mathbb{P}(w_t = w \mid w_{<t})$ over all tokens $w \in \mathcal{V}$. This formulation corresponds directly to the psychological *cloze test* (fill-in-the-blank evaluation), where an observer completes an incomplete prefix:

$$\text{“}\textit{This is a reasonable}\ _____\text{”} \implies \mathbb{P}(\text{sentence} \mid \text{This is a reasonable}) = ?$$

Crucially, this equivalence demonstrates that estimating local conditional probabilities across a vocabulary is mathematically sufficient to compute the global probability $\mathbb{P}(W)$ of any arbitrary document via Equation (**??**). Furthermore, text generation reduces to iteratively sampling from this local conditional distribution and appending each emitted token to the conditioning history. Consequently, solving conditional next-token prediction provides a complete framework for generative modeling.

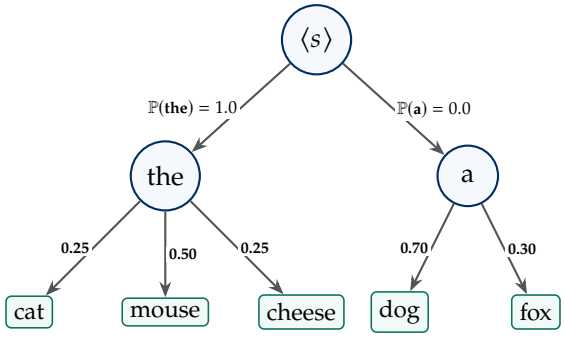


Figure 1.1: Visualizing autoregressive sequence generation as path traversal across a probability branching tree. At the root symbol $\langle s \rangle$, outgoing edges partition total probability mass across vocabulary tokens. At each node $w_{<t}$, children represent the conditional distribution $\mathbb{P}(w_t \mid w_{<t})$. The law of total probability guarantees that branch probabilities sum to unity at each decision point.

Geometrically, sequence generation can be interpreted as path traversal down a directed probability tree, as depicted in Figure **??**. At the root node $\langle s \rangle$, outgoing edges partition the total probability mass across the vocabulary $\mathcal{V}$. Selecting a token w_1 traverses an edge with probability $\mathbb{P}(w_1)$ to reach a new conditioning state, from which a new set of branches opens with conditional probabilities $\mathbb{P}(w_2 \mid w_1)$. The joint probability of an entire sentence equals the product of edge weights along that unique trajectory, and text generation corresponds to sampling a directed path from the root to a designated sequence-termination token $\langle /s \rangle$.

1.2 Markov assumptions and n-gram language models

Although Equation (**??**) is mathematically exact and preserves all syntactic and semantic dependencies, estimating $\mathbb{P}(w_t \mid w_{<t})$ directly from empirical frequencies presents an exponential combinatorial obstacle. As the sequence index t grows, the conditioning history $w_{<t} = (w_1, \ldots, w_{t-1})$ expands monotonically. Because each constituent token is drawn from a vocabulary $\mathcal{V}$ of size $V = |\mathcal{V}|$, the space of potential prefix histories preceding token t is the Cartesian product space $\mathcal{V}^{t-1}$, whose cardinality is:

$$|\mathcal{H}_t| = |\mathcal{V}|^{t-1} = V^{t-1}. \tag{1.3}$$

Even for a modest vocabulary of $V = 10{,}000$ words, tracking histories of length $t = 4$ requires estimating probabilities across $10{,}000^3 = 10^{12}$ distinct conditioning states. In any finite training corpus, almost every sequence of length greater than three occurs zero times.

To make estimation mathematically tractable, the principle of conditional independence posits that future tokens depend primarily on a bounded historical window. In his foundational 1948 treatise on information theory [**?**], Claude Shannon demonstrated that modeling natural language as a discrete Markov process produces increasingly coherent synthetic English as the conditioning order increases. Modern statistical treatments [**?**] formalize this simplification as the core of classical n-gram language modeling.

Definition 1.3: *Markov assumption of order $n-1$*

An n-gram language model assumes that the conditional probability of the current word w_t depends exclusively on the immediately preceding $n-1$ words, rendering it conditionally independent of earlier discourse history:

$$\mathbb{P}(w_t \mid w_1, w_2, \ldots, w_{t-1}) \approx \mathbb{P}(w_t \mid w_{t-n+1}, \ldots, w_{t-1}). \tag{1.4}$$

The integer $n \geq 1$ dictates the order of the model:

- **Unigram Model ($n = 1$, Order 0)**: Completely memoryless. Each word is treated as statistically independent of past and future tokens:

$$\mathbb{P}(w_1, \ldots, w_N) \approx \prod_{t=1}^{N} \mathbb{P}(w_t).$$

- **Bigram Model ($n = 2$, Order 1)**: First-order Markov chain. Conditioning depends exclusively on the single preceding token w_{t-1}:

$$\mathbb{P}(w_1, \ldots, w_N) \approx \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_{t-1}).$$

- **Trigram Model ($n = 3$, Order 2)**: Second-order Markov chain. Conditioning depends on the preceding pair of tokens (w_{t-2}, w_{t-1}):

$$\mathbb{P}(w_1, \ldots, w_N) \approx \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_{t-2}, w_{t-1}).$$

1.2.1 Combinatorial state-space reduction

The primary mathematical virtue of the Markov assumption is that it truncates an exponentially expanding state space into a fixed, stationary cardinality. Under an n-gram model, the conditioning context is always restricted to $n-1$ words, regardless of the absolute sequence position t:

$$|\mathcal{H}^{(n)}| = |\mathcal{V}|^{n-1} = V^{n-1}. \tag{1.5}$$

Consequently, specifying the complete transition probability distribution across the entire vocabulary requires an $(n-1)$-dimensional conditioning tensor containing:

$$P_{\text{params}} = V^{n-1} \times (V-1) \approx V^n, \tag{1.6}$$

free parameters. Table **??** contrasts the unconstrained historical state space with fixed n-gram models for realistic vocabulary sizes.

Table 1.1: State-space cardinality $|\mathcal{H}|$ as a function of sequence position t versus fixed n-gram Markov orders for vocabularies $V = 10{,}000$ and $V = 50{,}000$.

Model / Order	**Effective Context**	**Formula**	**States ($V = 10^4$)**	**States ($V = 5 \times 10^4$)**
Unconstrained ($t = 2$)	1 previous token	V^1	1.0×10^4	5.0×10^4
Unconstrained ($t = 3$)	2 previous tokens	V^2	1.0×10^8	2.5×10^9
Unconstrained ($t = 5$)	4 previous tokens	V^4	1.0×10^{16}	6.25×10^{18}
Unconstrained ($t = 10$)	9 previous tokens	V^9	1.0×10^{36}	1.95×10^{42}
Unconstrained ($t = 20$)	19 previous tokens	V^{19}	1.0×10^{76}	1.90×10^{89}
Unigram ($n = 1$)	0 tokens (memoryless)	$V^0 = 1$	1	1
Bigram ($n = 2$)	1 previous token	V^1	1.0×10^4	5.0×10^4
Trigram ($n = 3$)	2 previous tokens	V^2	1.0×10^8	2.5×10^9
4-gram ($n = 4$)	3 previous tokens	V^3	1.0×10^{12}	1.25×10^{14}

While the unconstrained state space exceeds 10^{76} states by step $t = 20$ (surpassing the order of magnitude of atoms in the observable universe), a bigram model maintains a fixed state space of 50,000 rows. However, this dimensional reduction introduces a severe theoretical trade-off: the model is conditionally blind to any token appearing outside the immediate $n-1$ window.

1.2.2 Maximum likelihood estimation (MLE)

Under Maximum Likelihood Estimation, we estimate the conditional transition probabilities directly by counting occurrences of exact token sequences in a training corpus [**?**]. For a generic n-gram:

$$\mathbb{P}_{\text{MLE}}(w_t \mid w_{t-n+1}, \ldots, w_{t-1}) = \frac{C(w_{t-n+1}, \ldots, w_{t-1}, w_t)}{C(w_{t-n+1}, \ldots, w_{t-1})}, \tag{1.7}$$

where $C(\cdot)$ denotes the raw frequency of that exact sequence in the training text. For a bigram model ($n = 2$), Equation (**??**) specializes to:

$$\mathbb{P}_{\text{MLE}}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t)}{C(w_{t-1})} = \frac{C(w_{t-1}, w_t)}{\sum_{w \in \mathcal{V}} C(w_{t-1}, w)}. \tag{1.8}$$

Normalizing each row by the unigram count of the conditioning prefix ensures that $\sum_{w \in \mathcal{V}} \mathbb{P}_{\text{MLE}}(w \mid w_{t-1}) = 1$.

1.3 Transition count matrix construction walkthrough

To observe the concrete mechanics of statistical n-gram estimation, we trace the extraction of bigram frequencies and transition probabilities from a controlled corpus comprising two sentences:

1. Sentence 1: *"The cat chased the mouse happily"*
2. Sentence 2: *"The mouse ate the cheese"*

Extracting all unique tokens (case-normalized with standard initial capitalization for clarity) yields a vocabulary of size $|\mathcal{V}| = 7$:

$$\mathcal{V} = \{\text{Ate, Cat, Chased, Cheese, Happily, Mouse, The}\}.$$

We construct the transition counts by scanning both sentences sequentially token by token. In Sentence 1, the observed transitions are (The $\rightarrow$ Cat), (Cat $\rightarrow$ Chased), (Chased $\rightarrow$ The), (The $\rightarrow$ Mouse), and (Mouse $\rightarrow$ Happily), each initialized with an empirical count of 1. In Sentence 2, we observe (The $\rightarrow$ Mouse) for a second time (incrementing $C(\text{The, Mouse})$ to 2), followed by (Mouse $\rightarrow$ Ate), (Ate $\rightarrow$ The), and (The $\rightarrow$ Cheese), each with count 1.

The row totals provide the unigram conditioning marginals $C(w_{t-1}) = \sum_w C(w_{t-1}, w)$. Here, the word *"The"* appears 4 times as a conditioning prefix (followed by *Cat* once, *Mouse* twice, and *Cheese* once), *"Mouse"* appears 2 times (followed by *Happily* and *Ate*), while *"Ate"*, *"Cat"*, and *"Chased"* each appear once. The words *"Cheese"* and *"Happily"* appear exclusively at the ends of sentences, yielding row marginals of zero since no subsequent tokens were observed in this corpus.

Applying Equation (**??**) yields the empirical transition probability matrix $\mathbf{M} \in \mathbb{R}^{7\times7}$ where row i represents condition w_{t-1} and column j represents next token w_t. Table **??** presents both raw counts and normalized MLE probabilities. The corresponding state transition graph is visualized in Figure **??**.

Table 1.2: Empirical transition count matrix and normalized bigram conditional probabilities $\mathbb{P}(w_t \mid w_{t-1})$ for vocabulary $\mathcal{V} = \{\text{Ate, Cat, Chased, Cheese, Happily, Mouse, The}\}$.

Condition (w_{t-1})	Ate	Cat	Chased	Cheese	Happily	Mouse	The	Row Total
Ate	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	1 (1.00)	1
Cat	0 (0.0)	0 (0.0)	1 (1.00)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	1
Chased	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	1 (1.00)	1
Cheese	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0
Happily	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0
Mouse	1 (0.50)	0 (0.0)	0 (0.0)	0 (0.0)	1 (0.50)	0 (0.0)	0 (0.0)	2
The	0 (0.0)	1 (0.25)	0 (0.0)	1 (0.25)	0 (0.0)	2 (0.50)	0 (0.0)	4

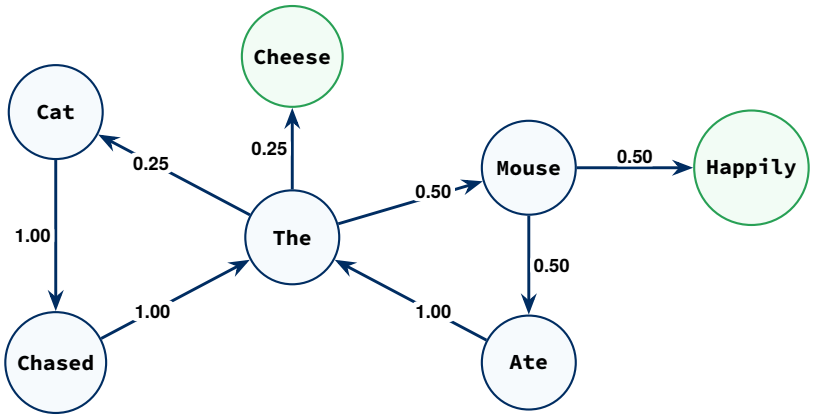


Figure 1.2: Markov state transition graph for the 7-token cat/mouse corpus. Directed edges indicate bigram transition probabilities $\mathbb{P}(w_t \mid w_{t-1})$. States `Cheese` and `Happily` (green) are terminal absorbing states with zero outgoing probability mass.

1.4 Autoregressive estimation and generation

With the transition matrix **M** constructed (Table **??**) and its state graph established (Figure **??**), the language model can execute two primary operational tasks: evaluating the likelihood of an arbitrary sequence of tokens, and generating synthetic text autoregressively.

1.4.1 Worked sentence probability calculation

Consider evaluating the joint probability of the sentence:

"the cat chased the cheese"

Under the first-order Markov assumption, the joint probability factors as:

$$\mathbb{P}(\text{the, cat, chased, the, cheese}) = \mathbb{P}(\text{the})\cdot\mathbb{P}(\text{cat} \mid \text{the})\cdot\mathbb{P}(\text{chased} \mid \text{cat})\cdot\mathbb{P}(\text{the} \mid \text{chased})\cdot\mathbb{P}(\text{cheese} \mid \text{the}). \tag{1.9}$$

Assuming generation begins deterministically with the initial token $\mathbb{P}(\text{the}) = 1.0$, we substitute the corresponding conditional probabilities from Table **??**:

$$\begin{aligned}
\mathbb{P}(\text{cat} \mid \text{the}) &= \frac{1}{4} = 0.25 \\
\mathbb{P}(\text{chased} \mid \text{cat}) &= 1.0 \\
\mathbb{P}(\text{the} \mid \text{chased}) &= 1.0 \\
\mathbb{P}(\text{cheese} \mid \text{the}) &= \frac{1}{4} = 0.25
\end{aligned}$$

Multiplying these values sequentially:

$$\mathbb{P}(\text{the, cat, chased, the, cheese}) = 1.0 \times \frac{1}{4} \times 1 \times 1 \times \frac{1}{4} = \frac{1}{16} = 0.0625. \tag{1.10}$$

Despite the sequence *"the cat chased the cheese"* never occurring as an entire sentence in the training corpus, the bigram model assigns it a strictly positive probability of 6.25%. By decomposing text into local constituent pairs, n-gram models exhibit rudimentary compositional generalization.

1.4.2 Autoregressive sequence generation and decoding dynamics

Autoregressive generation begins with an initial seed conditioning token, samples the next token from the resulting conditional categorical distribution, appends the sampled token to the history, and repeats the process sequentially.

To trace this mechanism concretely, consider initializing generation from the seed token *"the"* at step $t = 1$, where the conditioning history is history = ("the"). The active row from Table **??** yields the candidate distribution $\mathbb{P}(\text{cat} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{cheese} \mid \text{the}) = 0.25$, and $\mathbb{P}(\text{mouse} \mid \text{the}) = 0.50$. Assuming stochastic sampling selects **"mouse"**, the generated sequence expands to *"the mouse"*.

At step $t = 2$, conditioning on the updated state w_1 = "mouse" activates the conditional probabilities $\mathbb{P}(\text{ate} \mid \text{mouse}) = 0.50$ and $\mathbb{P}(\text{happily} \mid \text{mouse}) = 0.50$. When sampling selects **"ate"**, the prefix extends to *"the mouse ate"*. Subsequently, at step $t = 3$, the conditioning token *"ate"* exhibits only a single outgoing transition in the empirical corpus: $\mathbb{P}(\text{the} \mid \text{ate}) = 1.00$. The model therefore deterministically emits **"the"**, yielding *"the mouse ate the"*.

At step $t = 4$, conditioning once again on *"the"* reactivates the tripartite distribution over *cat* (0.25), *cheese* (0.25), and *mouse* (0.50). If sampling selects **"mouse"**, the generated sequence becomes *"the mouse ate the mouse"*. Because the first-order Markov model retains no memory beyond the immediate predecessor w_3 = "the", it cannot recognize that *"mouse"* was already

emitted two steps earlier. Finally, at step $t = 5$, conditioning on *"mouse"* offers equal probability between *ate* and *happily*. Sampling selects **"happily"**, producing the complete sequence *"the mouse ate the mouse happily"*. Because the token *"happily"* has zero outgoing transitions in Table **??**, the process reaches an absorbing terminal state and generation halts naturally.

This worked trace highlights two foundational behavioral characteristics of statistical Markov models:

Absorbing terminal states and sentence boundaries

In Table **??**, terminal tokens such as *"cheese"* and *"happily"* have a row count of zero ($C = 0$), leaving the conditional probability division $0/0$ mathematically undefined. If generation enters such a state, no further transitions can be sampled. In standard language modeling, this boundary condition is formalized by augmenting every training sentence with explicit boundary tokens: a beginning-of-sequence delimiter $\langle s \rangle$ and an end-of-sequence delimiter $\langle /s \rangle$ (or [EOS]). Under this convention, every word possesses a strictly defined distribution over either valid successor tokens or the terminal symbol $\langle /s \rangle$, guaranteeing a mathematically well-defined stopping criterion.

Cyclic limit cycles under greedy decoding

Because bigram models condition exclusively on the single preceding token w_{t-1}, deterministic decoding strategies such as greedy search ($\arg\max_w \mathbb{P}(w \mid w_{t-1})$) easily fall into repetitive limit cycles. In our transition graph, the path from *"the"* to *"mouse"* (0.50), from *"mouse"* to *"ate"* (0.50), and from *"ate"* back to *"the"* (1.00) forms a closed directed loop. Under deterministic selection, the model generates the infinite repetition *"the mouse ate the mouse ate the . . . "*. Modern foundation models mitigate repetitive degeneracy through extensive self-attention horizons, temperature scaling, nucleus sampling, and repetition penalties.

1.5 Structural limitations of statistical n-gram models: sparsity and context horizons

While n-gram models provided the foundation for speech recognition, statistical machine translation, and spelling correction throughout the 1980s and 1990s, they suffer from structural constraints that ultimately motivated the transition toward continuous neural architectures.

1.5.1 Combinatorial data sparsity and parameter explosion

To condition on $n-1$ tokens, an n-gram model requires maintaining a probability tensor of dimensionality $|\mathcal{V}|^n$. For realistic natural language vocabularies and software engineering codebases:

- With a modest vocabulary of $|\mathcal{V}| = 50{,}000$ unique tokens, a bigram model ($n = 2$) requires $|\mathcal{V}|^2 = 2.5 \times 10^9$ parameters (approximately 10 GB in 32-bit floating-point storage).
- A trigram model ($n = 3$) requires $|\mathcal{V}|^3 = 1.25 \times 10^{14}$ parameters, exceeding hundreds of terabytes.
- A 4-gram model ($n = 4$) escalates to $|\mathcal{V}|^4 = 6.25 \times 10^{18}$ parameters, requiring exabyte-scale memory.

Even if storage were unconstrained, the total count of distinct 4-grams in all digitized text is dwarfed by 10^{18}. Consequently, the empirical count tensor is overwhelmingly populated by zeros: the vast majority of valid linguistic phrases never appear in any finite training corpus.

1.5.2 The zero-frequency problem: joint probability and perplexity collapse

The most direct consequence of combinatorial sparsity is the zero-frequency problem. Under Maximum Likelihood Estimation, any n-gram that never appeared verbatim in the training text receives count $C = 0$, yielding:

$$\mathbb{P}_{\text{MLE}}(w_t \mid w_{t-n+1}^{t-1}) = \frac{C(w_{t-n+1}^{t})}{C(w_{t-n+1}^{t-1})} = 0. \tag{1.11}$$

By the probabilistic chain rule (Equation (**??**)), evaluating a test sequence $W_{\text{test}} = (w_1, \ldots, w_N)$ requires taking the product across all constituent conditional transitions:

$$\mathbb{P}(W_{\text{test}}) = \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_{<t}) = \mathbb{P}(w_1) \cdots 0 \cdots \mathbb{P}(w_N \mid w_{<N}) = 0. \tag{1.12}$$

A single unobserved token transition collapses the estimated joint probability of an entire document, article, or codebase to absolute zero.

Furthermore, when evaluating the model via Empirical Cross-Entropy $\mathcal{H}(W_{\text{test}})$ or Perplexity $\text{PPL}(W_{\text{test}})$:

$$\mathcal{H}(W_{\text{test}}) = -\frac{1}{N} \sum_{t=1}^{N} \log_2 \mathbb{P}(w_t \mid w_{<t}) \longrightarrow \infty, \quad \text{PPL}(W_{\text{test}}) = 2^{\mathcal{H}(W_{\text{test}})} \longrightarrow \infty. \tag{1.13}$$

An unobserved test n-gram results in infinite perplexity ($\text{PPL} = \infty$), rendering the entire language model unusable for empirical evaluation or benchmarking.

1.5.3 Finite context horizon limitations

An n-gram model is strictly blind to any token that occurred more than $n-1$ steps in the past:

$$\mathbb{P}(w_t \mid w_1, w_2, \ldots, w_{t-1}) \approx \mathbb{P}(w_t \mid w_{t-n+1}, \ldots, w_{t-1}). \tag{1.14}$$

Because the conditioning context is rigidly truncated to the preceding $n-1$ tokens, any semantic constraint, syntactic agreement, or discourse topic established earlier in the text is discarded. This finite context horizon causes severe failures across both natural language and source code.

In natural language, long-range syntactic agreements frequently span multiple clauses or modifier phrases separating a grammatical subject from its governing verb:

*"The **engineers** who designed the complex distributed compiler infrastructure [...] **were** vs **was** awarded the patent."*

In a 4-gram model ($n = 4$), predicting the verb depends exclusively on the three immediately preceding tokens: $\mathbb{P}(\text{verb} \mid \text{compiler, infrastructure, [...]})$. Because the singular noun *"infrastructure"* is adjacent to the verb position, local statistical associations favor the singular form *"was"*, violating subject-verb agreement with the true plural subject *"engineers"*.

In software engineering source code, this failure is even more pronounced due to strict lexical scoping and grammatical nesting. For example, a variable declared in a function signature:

```
void process_batch(std::vector<TransactionRequest>& transactions) { ...}
```

must be invoked using valid `std::vector` member methods (such as `.push_back()` or `.size()`) tens or hundreds of lines later. An n-gram model observing only the immediate prefix `transactions.` cannot access the original declaration, leading to invalid member access completions. Similarly, source code requires exact hierarchical pairing of delimiters (`{ ...}`, `[ ...]`, `( ...)`) and control flow blocks (`try ...catch ...finally`). Lacking an internal memory stack or unbounded context, n-gram models frequently generate mismatched delimiters or premature block closures.

1.5.4 Orthogonal discrete representations and lack of semantic generalization

In classical statistical NLP, words are treated as discrete, orthogonal atomic symbols. If the model observes:

*"The **cat** chased the mouse happily"*

it gains zero statistical confidence regarding the identical sentence containing synonyms:

*"The **feline** chased the mouse happily"*

Because *"cat"* and *"feline"* correspond to distinct row indices in Table **??**, observations of one do not inform the other. Discrete tables possess no intrinsic metric of semantic distance, synonymy, or conceptual overlap.

While long-range context degradation and semantic orthogonality ultimately require continuous neural representations (examined in Chapters **??** and **??**), the immediate mathematical obstacle in statistical language modeling is the zero-frequency problem ($C = 0 \implies \mathbb{P} = 0 \implies \text{PPL} = \infty$). To make n-gram models evaluable and robust against unseen sequences, we must redistribute probability mass from observed events to unobserved combinations. This redistribution is the objective of statistical smoothing.

1.6 Statistical smoothing and discounting algorithms

To prevent probability collapse ($\mathbb{P} = 0$) and infinite perplexity (PPL $= \infty$) on unseen test sequences, statistical language modeling relies on smoothing and discounting algorithms. The governing principle of smoothing is conservative probability reallocation: a fraction of probability mass is discounted from frequently observed events and redistributed across unseen combinations, ensuring that every valid sequence receives a strictly positive probability:

$$\forall w \in \mathcal{V}, \quad \mathbb{P}(w \mid \text{context}) > 0.$$

1.6.1 Laplace and additive smoothing

The simplest smoothing formulation, originated by Pierre-Simon Laplace, adds a uniform pseudo-count $\alpha > 0$ to every transition in the vocabulary. For an unconditioned unigram model, Laplace add-α smoothing assigns:

$$\mathbb{P}_{\text{Laplace}}(w) = \frac{C(w) + \alpha}{N + \alpha|\mathcal{V}|}, \tag{1.15}$$

where $N = \sum_{w \in \mathcal{V}} C(w)$ is the total token count. For a bigram model conditioned on w_{t-1}, the estimator modifies Equation (**??**) as:

$$\mathbb{P}_{\text{Laplace}}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t) + \alpha}{C(w_{t-1}) + \alpha|\mathcal{V}|}. \tag{1.16}$$

When $\alpha = 1$, the technique is known as *Laplace smoothing* (or Add-1); when $0 < \alpha < 1$, it is termed *Lidstone smoothing*.

We verify that Equation (**??**) defines a proper probability distribution satisfying the Kolmogorov axioms by summing over the entire vocabulary $\mathcal{V}$:

$$\begin{aligned}\sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{Laplace}}(w_t \mid w_{t-1}) &= \sum_{w_t \in \mathcal{V}} \frac{C(w_{t-1}, w_t) + \alpha}{C(w_{t-1}) + \alpha|\mathcal{V}|} \\ &= \frac{\sum_{w_t \in \mathcal{V}} C(w_{t-1}, w_t) + \sum_{w_t \in \mathcal{V}} \alpha}{C(w_{t-1}) + \alpha|\mathcal{V}|} \\ &= \frac{C(w_{t-1}) + \alpha|\mathcal{V}|}{C(w_{t-1}) + \alpha|\mathcal{V}|} = 1.\end{aligned} \tag{1.17}$$

Although mathematically well-formed, Add-1 smoothing performs poorly in practice. In realistic vocabularies where $|\mathcal{V}| = 50{,}000$, if word w_{t-1} occurred $C(w_{t-1}) = 100$ times, adding $\alpha|\mathcal{V}| = 50{,}000$ inflates the denominator from 100 to 50,100. A frequent observed transition with $C(w_{t-1}, w_t) = 50$ is diluted from an empirical probability of 0.50 to:

$$\frac{50 + 1}{100 + 50{,}000} = \frac{51}{50{,}100} \approx 0.00102.$$

Add-1 smoothing reallocates over 99.8% of the probability mass from observed events uniformly to unobserved transitions. Even Lidstone smoothing ($\alpha = 0.05$) assigns equal probability to all unseen words, failing to capture that some unseen words (such as frequent nouns) are inherently more probable than rare technical jargon.

1.6.2 Good-Turing discounting

Originally formulated in [**?**] during cryptographic research at Bletchley Park, *Good-Turing discounting* derives the probability of events seen r times directly from the count of events seen $r + 1$ times.

Let $r = C(w_{t-n+1}^{t}) \geq 0$ denote the empirical frequency of an n-gram. We define the *frequency of frequencies* N_r as the total number of distinct n-grams that appeared exactly r times in the training corpus:

$$N_r = |\{x : C(x) = r\}|. \tag{1.18}$$

The total number of tokens in the corpus is $N = \sum_{r=1}^{\infty} rN_r$, while N_0 represents the vast collection of valid n-grams that never appeared ($r = 0$).

Good-Turing replaces each raw count r with an effective discounted count r^*:

$$r^* = (r+1)\frac{N_{r+1}}{N_r}. \tag{1.19}$$

For any observed n-gram with count $r > 0$, its Good-Turing smoothed probability is:

$$\mathbb{P}_{\text{GT}}(w_t \mid w_{t-n+1}^{t-1}) = \frac{r^*}{N} = \frac{(r+1)N_{r+1}}{N \cdot N_r}. \tag{1.20}$$

The total probability mass reserved for all unobserved n-grams ($r = 0$) is determined by the proportion of *singletons* (N_1, words seen exactly once):

$$P_0 = \frac{N_1}{N}. \tag{1.21}$$

Because there are N_0 unobserved n-grams, each individual unseen n-gram receives probability:

$$\mathbb{P}_{\text{GT}}(\text{unseen}) = \frac{P_0}{N_0} = \frac{N_1}{N \cdot N_0}. \tag{1.22}$$

For large values of r, empirical counts become sparse: N_r fluctuates substantially or drops to zero, rendering the ratio N_{r+1}/N_r undefined. In production systems, *Simple Good-Turing* [**?**] replaces raw N_r values with smoothed estimates derived from a linear regression fit in log space ($\log N_r = a + b \log r$) for $r > k$.

1.6.3 Katz backoff with normalization

The recursive framework introduced in [**?**] extended Good-Turing discounting through a hierarchical backoff mechanism: if an n-gram has been observed, use its discounted probability; if it has never been observed, back off to the $(n-1)$-gram distribution, scaled by a normalizing factor.

For a bigram model, the Katz backoff estimator is:

$$\mathbb{P}_{\text{Katz}}(w_t \mid w_{t-1}) = \begin{cases} \mathbb{P}^*(w_t \mid w_{t-1}) & \text{if } C(w_{t-1}, w_t) > 0, \\ \alpha(w_{t-1})\mathbb{P}_{\text{Katz}}(w_t) & \text{if } C(w_{t-1}, w_t) = 0, \end{cases} \tag{1.23}$$

where $\mathbb{P}^*(w_t \mid w_{t-1}) = \frac{C^*(w_{t-1}, w_t)}{C(w_{t-1})}$ represents the discounted probability calculated using Good-Turing adjusted counts for low frequencies ($r \leq k$, typically $k = 5$) and MLE for high counts ($r > k$):

$$C^*(w_{t-1}, w_t) = \frac{\frac{r^*}{r} - \frac{(k+1)N_{k+1}}{N_1}}{1 - \frac{(k+1)N_{k+1}}{N_1}} \cdot r \quad \text{for } 1 \leq r \leq k. \tag{1.24}$$

The leftover probability mass discounted from observed bigrams is:

$$\beta(w_{t-1}) = 1 - \sum_{w_t : C(w_{t-1}, w_t) > 0} \mathbb{P}^*(w_t \mid w_{t-1}). \tag{1.25}$$

To ensure that $\sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{Katz}}(w_t \mid w_{t-1}) = 1$, the backoff weight $\alpha(w_{t-1})$ scales the lower-order unigram distribution over unobserved words:

$$\alpha(w_{t-1}) = \frac{\beta(w_{t-1})}{\sum_{w_t : C(w_{t-1}, w_t)=0} \mathbb{P}_{\text{Katz}}(w_t)}. \tag{1.26}$$

Summing across the entire vocabulary confirms proper normalization:

$$\begin{aligned} \sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{Katz}}(w_t \mid w_{t-1}) &= \sum_{C>0} \mathbb{P}^*(w_t \mid w_{t-1}) + \sum_{C=0} \alpha(w_{t-1}) \mathbb{P}_{\text{Katz}}(w_t) \\ &= \big(1 - \beta(w_{t-1})\big) + \alpha(w_{t-1}) \sum_{C=0} \mathbb{P}_{\text{Katz}}(w_t) \\ &= \big(1 - \beta(w_{t-1})\big) + \beta(w_{t-1}) = 1. \end{aligned} \tag{1.27}$$

1.6.4 Kneser-Ney smoothing and continuation probabilities

The standard of statistical language modeling is *Kneser-Ney smoothing* [**?**], which introduces two key innovations: absolute discounting and continuation probabilities.

Empirical analysis of corpus frequencies demonstrates that subtracting a fixed constant $d \in (0, 1)$ (typically $d \approx 0.75$) from all positive counts accurately models the discounted frequency:

$$C_{\text{discounted}} = \max\left(C(w_{t-n+1}^{t}) - d,\, 0\right).$$

However, standard backoff mechanisms encounter a subtle failure mode when falling back to unigram distributions. Consider predicting the missing word in the sentence:

"I can't see without my reading _____*"*

The word *"glasses"* should receive a high conditional probability. Now consider the word *"Francisco"*. In standard text corpora, *"Francisco"* has a high unigram frequency because the proper noun *"San Francisco"* appears frequently. Under standard Katz backoff, because the bigram *"reading Francisco"* has zero count, the model backs off to the high unigram frequency of *"Francisco"*, assigning it an inappropriately high probability of completing the sentence.

The insight in [**?**] recognized that the unigram backoff distribution should not reflect *how often* a word appears overall, but *how versatile* the word is across diverse preceding contexts.

Definition 1.4: *Continuation probability*

The *continuation probability* $\mathbb{P}_{\text{cont}}(w_t)$ is the ratio of the number of unique preceding word types that precede w_t to the total number of unique bigram types in the vocabulary:

$$\mathbb{P}_{\text{cont}}(w_t) = \frac{|\{w_{t-1} : C(w_{t-1}, w_t) > 0\}|}{\sum_{w'} |\{w_{t-1} : C(w_{t-1}, w') > 0\}|} = \frac{|\{w_{t-1} : C(w_{t-1}, w_t) > 0\}|}{|\{(w_{t-1}, w_t) : C(w_{t-1}, w_t) > 0\}|}. \tag{1.28}$$

Because *"Francisco"* almost exclusively appears preceded by the single word *"San"*, $|\{w_{t-1} : C(w_{t-1}, \text{Francisco}) > 0\}| \approx 1$, resulting in a tiny continuation probability. In contrast, words like *"glasses"* or *"book"* appear in dozens of distinct contexts, yielding high continuation probabilities.

Combining absolute discounting with continuation probabilities yields the *Interpolated Kneser-Ney* estimator:

$$\mathbb{P}_{\text{KN}}(w_t \mid w_{t-n+1}^{t-1}) = \frac{\max\left(C(w_{t-n+1}^{t}) - d,\, 0\right)}{C(w_{t-n+1}^{t-1})} + \lambda(w_{t-n+1}^{t-1}) \mathbb{P}_{\text{KN}}(w_t \mid w_{t-n+2}^{t-1}), \tag{1.29}$$

where $\lambda(w_{t-n+1}^{t-1})$ is an explicit normalizing interpolation weight:

$$\lambda(w_{t-n+1}^{t-1}) = \frac{d}{C(w_{t-n+1}^{t-1})} \cdot \left|\{w : C(w_{t-n+1}^{t-1}, w) > 0\}\right|. \tag{1.30}$$

For a bigram model ($n = 2$), the base recursion terminates at the unigram continuation probability:

$$\mathbb{P}_{\text{KN}}(w_t \mid w_{t-1}) = \frac{\max\left(C(w_{t-1}, w_t) - d,\, 0\right)}{C(w_{t-1})} + \frac{d}{C(w_{t-1})} \cdot |\{w : C(w_{t-1}, w) > 0\}| \cdot \mathbb{P}_{\text{cont}}(w_t). \tag{1.31}$$

Summing Equation (**??**) over all $w_t \in \mathcal{V}$ verifies proper normalization:

$$\begin{aligned}
\sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{KN}}(w_t \mid w_{t-1}) &= \sum_{w_t : \text{C}>0} \frac{C(w_{t-1}, w_t) - d}{C(w_{t-1})} + \lambda(w_{t-1}) \sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{cont}}(w_t) \\
&= \frac{\sum_{\text{C}>0} C(w_{t-1}, w_t) - d \cdot |\{w : C(w_{t-1}, w) > 0\}|}{C(w_{t-1})} + \lambda(w_{t-1}) \cdot 1 \\
&= \frac{C(w_{t-1}) - d|\{w : C(w_{t-1}, w) > 0\}|}{C(w_{t-1})} + \frac{d|\{w : C(w_{t-1}, w) > 0\}|}{C(w_{t-1})} = 1.
\end{aligned} \tag{1.32}$$

Interpolated Kneser-Ney consistently outperforms other statistical smoothing techniques, serving as the industry standard until the advent of deep continuous language models.

With competing smoothing algorithms established—each reallocating probability mass under different statistical assumptions—we require objective, quantitative criteria to evaluate and compare models on held-out test data. This necessitates formal information-theoretic evaluation metrics.

1.7 Evaluation metrics: cross-entropy and perplexity

To benchmark and compare different language models quantitatively without executing costly downstream applications, natural language processing relies on intrinsic, information-theoretic metrics: *Cross-Entropy* and *Perplexity* [**?** **?**].

1.7.1 Information-theoretic foundations: entropy and KL divergence

Let $\mathcal{X}$ denote a discrete sample space (the vocabulary $\mathcal{V}$), and let $P(x)$ represent the true, underlying, but inaccessible probability distribution of human language.

Definition 1.5: *Shannon entropy*

The *Shannon entropy* $H(P)$ measures the fundamental, irreducible uncertainty or average information content inherent in distribution P [**?**]:

$$H(P) = -\sum_{x \in \mathcal{X}} P(x) \log_2 P(x) = \mathop{\mathbb{E}}_{x \sim P} \left[\log_2 \frac{1}{P(x)} \right]. \tag{1.33}$$

By Shannon's source coding theorem, $H(P)$ represents the absolute theoretical lower bound on the expected number of bits required to encode a symbol emitted by P without loss.

When constructing a language model, we approximate the true distribution P with an estimated parameterized distribution Q_θ. To quantify the statistical discrepancy between the true distribution P and our approximating model Q_θ, we employ the *Kullback-Leibler (KL) divergence*.

Definition 1.6: *Kullback-Leibler (KL) divergence*

The *KL divergence* (or relative entropy) from model distribution Q to target distribution P is:

$$D_{\mathrm{KL}}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log_2 \left(\frac{P(x)}{Q(x)} \right) = \mathop{\mathbb{E}}_{x \sim P} \left[\log_2 \frac{P(x)}{Q(x)} \right]. \tag{1.34}$$

Theorem 1.1: *Gibbs' inequality*

For any two discrete probability distributions P and Q, the KL divergence is non-negative:

$$D_{\mathrm{KL}}(P \parallel Q) \geq 0, \tag{1.35}$$

with equality $D_{\mathrm{KL}}(P \parallel Q) = 0$ if and only if $P(x) = Q(x)$ for all $x \in \mathcal{X}$.

We verify Gibbs' inequality by expressing the divergence via the natural logarithm and

utilizing the inequality $\ln(u) \le u - 1$ for all $u > 0$ (with equality if and only if $u = 1$):

$$\begin{aligned} -D_{\mathrm{KL}}(P \parallel Q) &= \sum_{x \in \mathcal{X}} P(x) \log_2 \left(\frac{Q(x)}{P(x)} \right) = \frac{1}{\ln 2} \sum_{x \in \mathcal{X}} P(x) \ln \left(\frac{Q(x)}{P(x)} \right) \\ &\le \frac{1}{\ln 2} \sum_{x \in \mathcal{X}} P(x) \left(\frac{Q(x)}{P(x)} - 1 \right) \\ &= \frac{1}{\ln 2} \left(\sum_{x \in \mathcal{X}} Q(x) - \sum_{x \in \mathcal{X}} P(x) \right) = \frac{1}{\ln 2} (1 - 1) = 0. \end{aligned}$$

Multiplying both sides by -1 reverses the inequality, yielding $D_{\mathrm{KL}}(P \parallel Q) \ge 0$. ■

1.7.2 Cross-entropy decomposition

Expanding the logarithm in the definition of KL divergence demonstrates the fundamental relationship connecting Entropy, KL Divergence, and Cross-Entropy:

$$\begin{aligned} D_{\mathrm{KL}}(P \parallel Q) &= \sum_{x \in \mathcal{X}} P(x) \log_2 P(x) - \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) \\ &= -H(P) + \left(- \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) \right). \end{aligned} \tag{1.36}$$

Rearranging terms defines the *Cross-Entropy* $H(P, Q)$:

$$H(P, Q) \triangleq - \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) = H(P) + D_{\mathrm{KL}}(P \parallel Q). \tag{1.37}$$

Equation (**??**) provides the theoretical justification for cross-entropy minimization during training. Because the true language distribution P is determined by human linguistic behavior, its entropy $H(P)$ is an invariant physical constant. When minimizing cross-entropy with respect to model parameters θ:

$$\arg\min_{\theta} H(P, Q_\theta) = \arg\min_{\theta} \left(H(P) + D_{\mathrm{KL}}(P \parallel Q_\theta) \right) = \arg\min_{\theta} D_{\mathrm{KL}}(P \parallel Q_\theta). \tag{1.38}$$

Minimizing cross-entropy is mathematically identical to driving the KL divergence between model Q_θ and the true language distribution P to zero.

1.7.3 Empirical sequence cross-entropy

Because we do not have closed-form access to the true distribution $P(x)$, we cannot compute the expectation $\mathbb{E}_{x \sim P}[\cdot]$ directly. Instead, by the Shannon-McMillan-Breiman theorem (the ergodic theorem for stationary information sources), the expected value is estimated empirically by sampling a long test sequence $W = (w_1, w_2, \ldots, w_T) \sim P$:

$$\mathcal{H}(W) = -\frac{1}{T} \log_2 Q(w_1, w_2, \ldots, w_T) = -\frac{1}{T} \sum_{t=1}^{T} \log_2 Q(w_t \mid w_{<t}). \tag{1.39}$$

Empirical cross-entropy measures the average number of bits required to compress each token of test text using model Q.

1.7.4 Perplexity as an intrinsic evaluation metric

While cross-entropy evaluates performance on a logarithmic scale, researchers and practitioners report *Perplexity* (PPL), defined as the exponentiated cross-entropy loss:

$$\mathrm{PPL}(W) = 2^{\mathcal{H}(W)} = 2^{-\frac{1}{T}\sum_{t=1}^{T}\log_2 Q(w_t|w_{<t})} \tag{1.40}$$

$$= \left(\prod_{t=1}^{T}\frac{1}{Q(w_t \mid w_{<t})}\right)^{\frac{1}{T}}. \tag{1.41}$$

Perplexity represents the reciprocal geometric mean of the token probabilities assigned by the model across the test corpus.

1.7.5 Formal derivation: perplexity as the effective branching factor

The intuitive interpretation of perplexity is that it quantifies the *effective branching factor* of a language model. We formalize this property through the following derivation:

Theorem 1.2: *Perplexity of a uniform K-ary distribution*

Let a language model Q predict next tokens such that at each timestep t, the probability mass is distributed uniformly across exactly K equally likely candidate tokens ($Q(w_t \mid w_{<t}) = \frac{1}{K}$ for each candidate). Then the perplexity on any generated sequence W of length T is identically K:

$$\mathrm{PPL}(W) = K. \tag{1.42}$$

Substituting the uniform prediction probability $Q(w_t \mid w_{<t}) = \frac{1}{K}$ into the empirical cross-entropy formula yields:

$$\mathcal{H}(W) = -\frac{1}{T}\sum_{t=1}^{T}\log_2\left(\frac{1}{K}\right) = -\frac{1}{T}\sum_{t=1}^{T}\left(-\log_2 K\right)$$
$$= \frac{1}{T}\left(T\log_2 K\right) = \log_2 K.$$

Exponentiating the cross-entropy:

$$\mathrm{PPL}(W) = 2^{\mathcal{H}(W)} = 2^{\log_2 K} = K. \quad \blacksquare$$

When a model evaluates a text corpus and achieves PPL = 42, this indicates that, on average, the model's predictive uncertainty at each token is mathematically equivalent to choosing uniformly at random from a shortlist of 42 equally plausible words. A uniform random guesser across vocabulary $\mathcal{V}$ achieves PPL = $|\mathcal{V}|$ (for example, PPL = 50,000), while an optimal oracle that predicts every token with probability 1.0 achieves the theoretical lower bound PPL = 1.0. State-of-the-art foundation models on natural language benchmarks typically achieve perplexities between 8 and 15.

1.7.6 Logarithmic base conversions: bits versus nats

A practical mathematical distinction arises when connecting information-theoretic formulations (which employ base-2 logarithms) with deep learning frameworks such as PyTorch (which compute natural base-e cross-entropy loss):

Property	Information Theory (Bits)	Deep Learning / PyTorch (Nats)
Loss Formula	$\mathcal{H}_2 = -\frac{1}{T}\sum \log_2 Q(w_t \mid w_{<t})$	$\mathcal{L}_{\text{CE}} = -\frac{1}{T}\sum \ln Q(w_t \mid w_{<t})$
Unit	Bits per token	Nats per token
Perplexity Formula	$\text{PPL} = 2^{\mathcal{H}_2}$	$\text{PPL} = \exp(\mathcal{L}_{\text{CE}}) = e^{\mathcal{L}_{\text{CE}}}$
Conversion Identity	$\mathcal{H}_2 = \frac{\mathcal{L}_{\text{CE}}}{\ln 2} \approx \frac{\mathcal{L}_{\text{CE}}}{0.693147}$,	$\mathcal{L}_{\text{CE}} = \mathcal{H}_2 \cdot \ln 2 \approx 0.693147 \cdot \mathcal{H}_2$

To observe the effect of this conversion, suppose a model trained on code completion reports an evaluation loss of $\mathcal{L}_{\text{CE}} = 2.7726$ nats. Computing base-e perplexity directly yields:

$$\text{PPL} = e^{2.7726} = \exp(2.7726) \approx 16.00.$$

Converting the loss to information bits produces:

$$\mathcal{H}_2 = \frac{2.7726}{\ln 2} = \frac{2.7726}{0.693147} = 4.00 \text{ bits/token},$$

which confirms the identical perplexity value: $\text{PPL} = 2^{\mathcal{H}_2} = 2^{4.00} = 16.00$. Inadvertently evaluating $2^{\mathcal{L}_{\text{CE}}} = 2^{2.7726} \approx 6.83$ would severely underestimate the true branching factor by more than 57%, highlighting the necessity of matching logarithmic bases.

1.7.7 The structural ceiling of smoothed n-gram models

While Kneser-Ney smoothing substantially reduces test perplexity by eliminating zero-frequency collapses, smoothed n-gram models remain constrained by fundamental structural limitations:

- **Context Horizon Truncation**: Conditioning remains restricted to $n-1$ tokens. Increasing $n \geq 5$ yields diminishing returns because higher-order frequencies are almost entirely unobserved, causing the model to constantly back off to lower orders.
- **Absence of Parameter Sharing**: Words remain discrete orthogonal indices. The model cannot generalize that *"The programmer wrote a test"* informs *"The developer authored a test"*.
- **Combinatorial Storage Overheads**: Count matrices for large corpora require gigabytes to terabytes of memory.

These structural limits motivated researchers to explore paradigms that replace discrete counting tables with continuous, parameterized functions.

1.8 Historical evolution of language models

Understanding modern large language models requires appreciating the 75-year evolutionary trajectory that progressively addressed the mathematical and computational barriers of discrete symbol counting. Figure **??** synthesizes the macro-historical progression of language modeling from Claude Shannon's foundational 1948 information theory to modern autonomous software engineering systems.

1.8.1 1948–1980s: Shannon's information theory and Markov n-grams

The conceptual origin of language modeling stems from Claude Shannon's landmark 1948 paper *"A Mathematical Theory of Communication"* [**?**]. In that work, written language was modeled as a discrete stochastic process generated by an underlying stationary Markov source over a finite alphabet.

Shannon introduced the *Shannon game*—a thought experiment in which an observer predicts the next letter in an unknown sequence given preceding characters. Shannon demonstrated that as the conditioning context length increases from zero-order (uniform random selection) to unigram (letter frequencies), bigram (character pairs), and trigram (triplets), generated text converges progressively toward natural English orthography and syntax.

In this framework, the conditional probability of token w_t given context $w_1, \ldots, w_{t-1}$ is simplified via an order-$(n-1)$ Markov assumption, as illustrated in Figure **??** for programming language tokens.

1.8.2 1990s: statistical decision trees and context partitioning

Throughout the 1990s, as speech recognition and machine translation systems expanded, flat n-gram tables could not accommodate long histories without exponential parameter growth. To address this limitation, *decision-tree language models* were developed in [**?**]. Rather than conditioning on an exact sequence of $n-1$ tokens, a binary decision tree poses a hierarchy of lexical and syntactic queries regarding the history $\mathcal{H} = (w_{t-1}, w_{t-2}, \ldots)$.

As shown in Figure **??**, context histories that exhibit similar predictive patterns are pooled into the same leaf node $\mathcal{L}_k$, yielding distributions $\mathbb{P}(w_t \mid \mathcal{L}_k)$. While this reduced free parameters relative to full combinatorial tables, the architecture remained reliant on discrete categorical partitions without smooth semantic interpolation.

1.8.3 2000–2003: The neural probabilistic language model

In the seminal 2003 paper *"A Neural Probabilistic Language Model"* [**?**], Yoshua Bengio and colleagues introduced continuous distributed representations to alleviate the curse of dimensionality.

The neural probabilistic formulation in [**?**] replaced discrete count tables with a continuous parameterized architecture (Figure **??**):

1. **Continuous Embedding Projection**: Each word $w \in \mathcal{V}$ is associated with a real-valued distributed feature vector $\mathbf{c}_w \in \mathbb{R}^d$ (where $d \ll |\mathcal{V}|$, e.g., $d = 100$). The vocabulary lookup matrix $\mathbf{C} \in \mathbb{R}^{|\mathcal{V}| \times d}$ is learned jointly via backpropagation.

2. **Semantic Smoothing via Continuous Space**: If *"cat"* and *"feline"* share similar embedding vectors ($\mathbf{c}_{\text{cat}} \approx \mathbf{c}_{\text{feline}}$), observing *"the cat chased the mouse"* automatically increases the model's confidence in *"the feline chased the mouse"*. Continuous geometry overcomes the orthogonal symbol limitation.

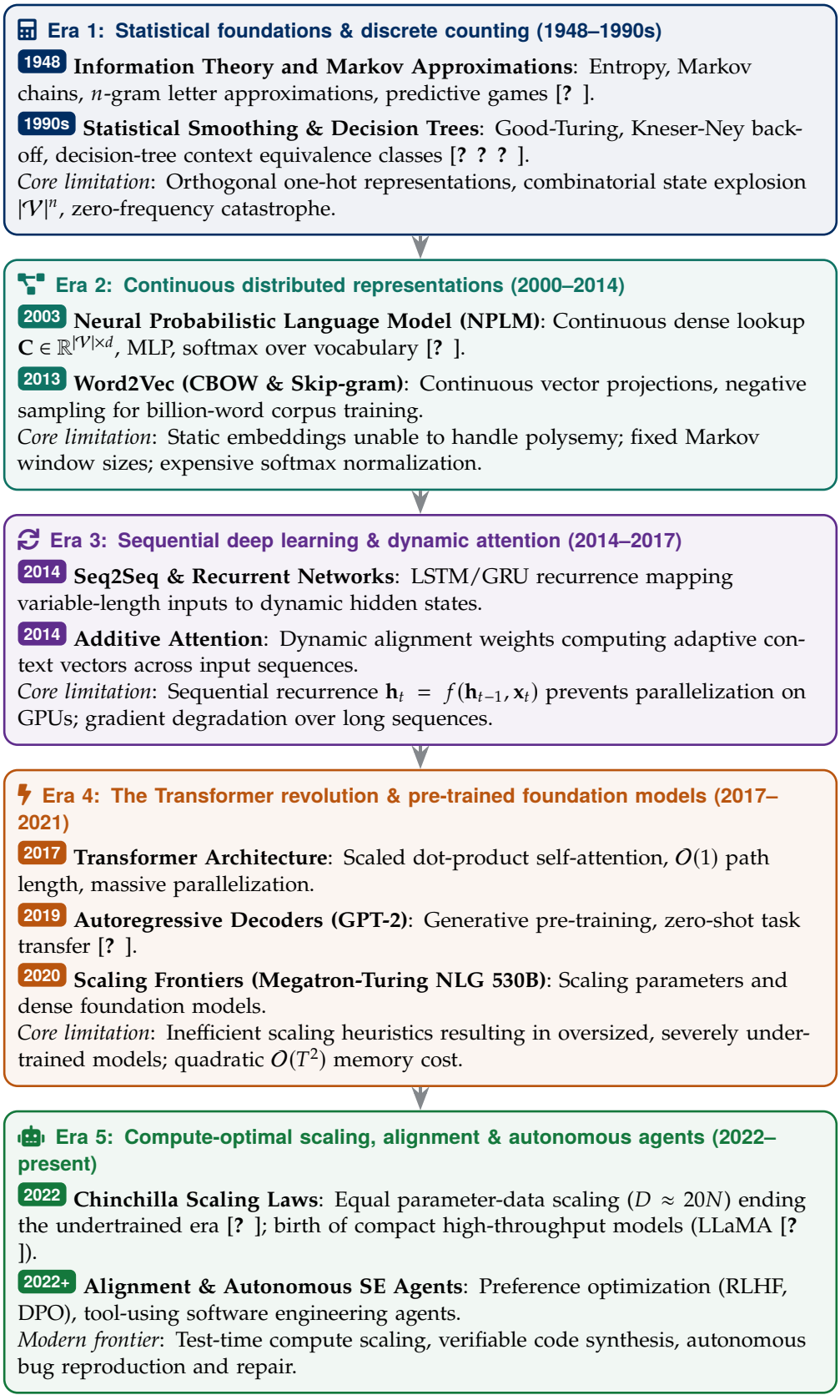


Figure 1.3: Chronological timeline of language model evolution from 1948 to the present.

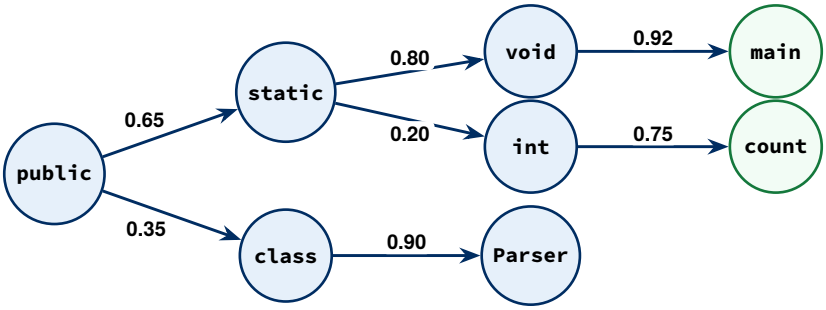


Figure 1.4: Markov chain state transition graph for software tokens.

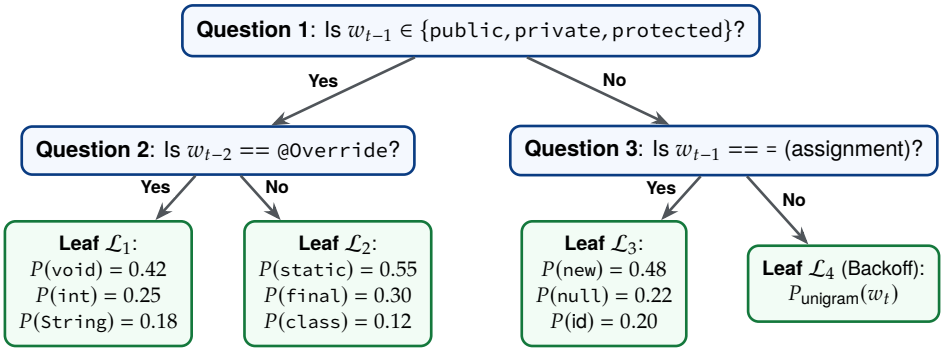


Figure 1.5: Decision tree context partitioning for conditional language modeling [**?**].

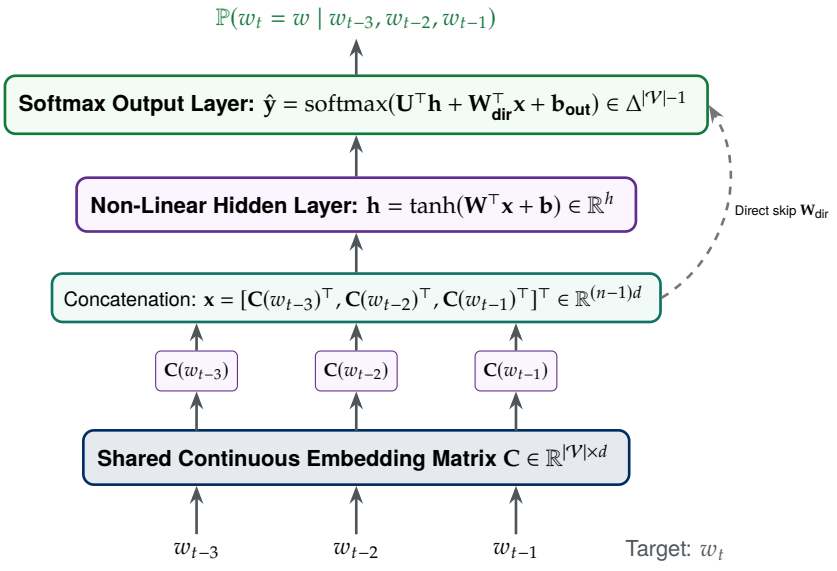


Figure 1.6: The Neural Probabilistic Language Model (NPLM) architecture [**?**]. Discrete context words are mapped into continuous dense feature vectors via shared matrix $\mathbf{C} \in \mathbb{R}^{|\mathcal{V}| \times d}$, concatenated, and processed through a non-linear hidden layer to predict next-token probabilities via softmax.

3. **Softmax Output Normalization**: The network projects hidden activations to $|\mathcal{V}|$ unnormalized logits and applies the softmax function to guarantee a valid probability distribution:

$$\mathbb{P}(w_t = i \mid w_{<t}) = \frac{\exp(z_i)}{\sum_{j=1}^{|\mathcal{V}|} \exp(z_j)}. \tag{1.43}$$

1.8.4 2018–2019: Generative pre-training and GPT-2

While continuous representations were established in [**?**], models throughout the 2000s and early 2010s remained constrained in parameter scale and context length. In 2018–2019, the GPT-2 architecture was introduced in [**?**] (*"Language Models are Unsupervised Multitask Learners"*).

GPT-2 demonstrated that autoregressive decoder-only models trained on hundreds of gigabytes of uncurated web text could perform diverse natural language and software engineering tasks without task-specific supervised fine-tuning. By predicting next tokens across context windows of 1,024 tokens, the model exhibited emergent zero-shot transfer across reading comprehension, machine translation, and Python code generation.

1.8.5 2020–2022: Megatron-Turing NLG 530B and compute-optimal scaling

Following GPT-2 and GPT-3, deep learning research prioritized aggressive model scaling, culminating in dense models such as Megatron-Turing NLG 530B [**?**]. However, early power-law scaling heuristics [**?**] recommended scaling parameters faster than training data. Consequently, models such as Megatron-Turing NLG 530B and GPT-3 (175B parameters trained on 300B tokens) were undertrained relative to their computational capacity.

In 2022, the study *"Training Compute-Optimal Large Language Models"* [**?**] established the **Chinchilla** scaling laws:

$$D \approx 20 \times N, \tag{1.44}$$

where N denotes parameter count and D represents training token volume. For compute optimality, model size and training data must scale in equal proportion. This insight initiated the current era of compact, thoroughly trained open-weight models (e.g., LLaMA [**?**] trained on trillions of tokens), improving inference efficiency in production and software engineering environments.

1.8.6 From discrete counting tables to continuous parametric learners: bridge to deep learning

The formal trajectory examined throughout this chapter highlights two structural constraints inherent in discrete language modeling:

- **The Discrete Counting Limit**: Exact probability factorization via the chain rule of probability is mathematically unconstrained, yet empirical n-gram tables suffer from exponential state-space growth (V^n), rigid context truncation ($n-1$), and orthogonal symbol representation.
- **The Smoothing Ceiling**: While statistical smoothing methods prevent zero-frequency collapse ($\mathbb{P} = 0 \implies \text{PPL} = \infty$), they operate solely by reallocating probability mass across discrete table entries. Consequently, they cannot capture non-linear feature interactions or transfer statistical strength between semantically related tokens.

To overcome the discrete counting paradigm, natural language processing transitioned from frequency tables to continuous, parameterized neural networks:

$$f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{W}^\top \mathbf{x} + \mathbf{b}. \tag{1.45}$$

Rather than counting token co-occurrences directly, modern language modeling parameterizes estimators as continuous differentiable functions whose weights $\boldsymbol{\theta}$ are optimized systematically through gradient descent and backpropagation, establishing the computational framework developed in subsequent chapters.